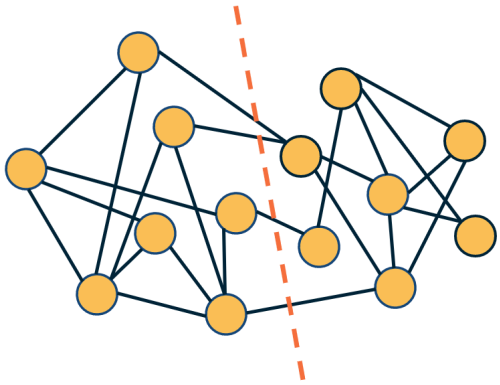
# L7: The Graph Partitioning Problem

Let us start with ***"graph partitioning"*** – a classical problem in computer science.



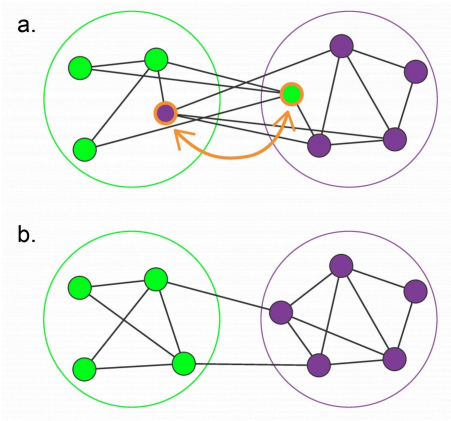
Given a graph, how would we partition the nodes into two non-overlapping sets of the same size so that we minimize the number of edges between the two sets? This is also known as the ***"minimum bisection problem".***

The visualization at the right shows such a bisection. Note that there are only four edges that cross the partition boundary *(red dashed line)* – and each set in the partition has seven nodes.

We can also state more general versions of this problem in which we partition the network into *K* non-overlapping sets of the same size, where *K>2* is a given constant.

The graph partitioning problem is important in many applications. For instance, in distributed computing, we are given a program in which there are *N* interacting threads but we only have *K* processors (*K<N*). The interactions between threads can be represented with a graph, where each edge represents a pair of threads that need to communicate while processing. It is important to assign interacting groups of threads to the same processor (so that we minimize the inter-processor communication delays) and to also equally split the threads between the *K* processors so that their load is balanced.

*Figure from Box 9.1 of Network Science Book by A.L.Barabási*




The graph partitioning problem is NP-Complete and so we only have efficient algorithms that can approximate its solution. The **Kerninghan-Lin algorithm** – as shown in the visualization above -- iteratively switches one pair of nodes between the two sets of the partition, selecting the pair that will cause the largest reduction in the number of edges that cut across the partition.

For our purposes, it is important to note that in the graph partitioning problem we are given the number of sets in the partition and that each set should have the same size. As we will see, this is very different than the community detection problem.